

Ciclo de vida - API .Net

Resumo

O desenvolvimento de aplicações baseadas em APIs é um dos pilares da arquitetura de software moderna, sendo amplamente utilizado em sistemas distribuídos, aplicações web e serviços integrados. Nesse contexto, compreender o funcionamento interno dessas aplicações é essencial para o desenvolvimento de soluções escaláveis, performáticas e de fácil manutenção.

Este trabalho tem como objetivo apresentar e detalhar o ciclo de vida de uma API desenvolvida em .NET, abordando desde a inicialização da aplicação até o processamento completo de uma requisição. São explorados conceitos fundamentais como o modelo de hospedagem, configuração do pipeline de middlewares, funcionamento do servidor web *Kestrel*, criação do contexto de requisição (*HttpContext*) e execução de endpoints.

Além disso, o trabalho aprofunda o funcionamento da injeção de dependência, destacando o papel do contêiner de serviços, os diferentes tempos de vida das dependências e o gerenciamento automático de recursos. Também são apresentados os mecanismos de execução assíncrona por meio de *async/await*, evidenciando sua importância para a escalabilidade e eficiência no processamento de requisições.

Por fim, é demonstrado de forma estruturada o fluxo completo de uma requisição em uma API .NET, integrando todos os conceitos apresentados e evidenciando como o framework gerencia de forma eficiente o ciclo de vida da aplicação e das requisições.

Palavras-chave: API, .NET, ASP.NET Core, Injeção de Dependência, Middleware, Execução Assíncrona.

Lista de ilustrações

Figura 1 – Fluxo de inicialização de uma API .NET até o método <code>Run()</code>	9
Figura 2 – <i>Singleton</i> - Clientes podem não se dar conta que estão lidando com o mesmo objeto a todo momento (Refactoring.Guru, 2025).	14
Figura 3 – Envio de requisição HTTP ao endpoint <i>cardcomment</i> da API Ordem utilizando o método <i>POST</i>	21
Figura 4 – Fluxo completo do ciclo de vida de uma requisição em uma API .NET. . . .	32

Sumário

1	INTRODUÇÃO	5
2	CICLO DE VIDA DA APLICAÇÃO	6
2.1	Síntese do capítulo	10
3	INJEÇÃO DE DEPENDÊNCIA NO .NET	12
3.1	Conceito e Contêiner de Serviços (<i>Service Container</i>)	12
3.2	Ciclo de Vida das Dependências	13
3.2.1	<i>Singleton</i>	14
3.2.2	<i>Scoped</i>	15
3.2.3	<i>Transient</i>	16
3.3	<i>Disposal of Services</i>	16
3.4	Síntese do capítulo	17
4	CICLO DE VIDA DA REQUISIÇÃO	18
4.1	Conceito de requisição em aplicações web	18
4.2	Recebimento da requisição pelo servidor	19
4.3	Pipeline de middlewares	22
4.4	Resolução de dependências durante a requisição	24
4.5	Execução assíncrona no ASP.NET Core	27
4.6	Finalização da requisição e descarte de recursos	29
4.7	Fluxo completo de uma requisição	30
	REFERÊNCIAS	33

1 Introdução

O desenvolvimento baseado em APIs é um dos pilares da arquitetura de software moderna, especialmente em cenários que envolvem microsserviços, sistemas distribuídos, aplicações móveis e também aplicações web ([ALMEIDA, 2025](#)). Hoje, grande parte dos sistemas utilizam esse modelo de software, resultando em um grande investimento por parte das empresas em profissionais que trabalham com essa tecnologia. Nesse contexto, é de suma importância entender o funcionamento interno dessas aplicações para criação de soluções escaláveis, performáticas, seguras e com boa manutenibilidade.

No ecossistema do .net, as APIs são amplamente utilizadas para expor funcionalidades por meio do protocolo HTTP, seguindo padrões como o próprio REST. Apesar da aparente simplicidade na utilização de frameworks, como o próprio ASP.NET Core, o comportamento interno de uma API envolve particularidades e etapas que ocorrem desde o momento que ela é inicializada até o processamento completo de uma requisição ([Microsoft, 2025c](#)). Esse conjunto de etapas é conhecido como ciclo de vida de uma aplicação e ciclo de vida de uma requisição.

O ciclo de vida da aplicação (API lifecycle) compreende todos os processos executados durante a inicialização da API, incluindo o carregamento de suas configurações, criação do host, inicialização dos middlewares e definições das injeções de dependências. Enquanto isso, o ciclo de vida de uma requisição descreve o fluxo percorrido por cada chamada http, incluindo a chegada da chamada no servidor, até a resposta final ao cliente, passando pelos processos de roteamento, processamentos de middlewares, execução de endpoints e serialização dos dados ([Microsoft, 2025b](#)).

Um dos componentes centrais que conectam esses dois ciclos é o mecanismo de injeção de dependência (Dependency Injection – DI). A DI permite a criação e o gerenciamento de dependências de forma desacoplada, promovendo princípios como baixo acoplamento, alta coesão e testabilidade. Além disso, os diferentes tempos de vida das dependências (singleton, scoped e transient) estão diretamente relacionados ao ciclo de vida da aplicação e ao escopo de cada requisição.

Dessa forma, compreender o ciclo de vida de uma API .NET não se limita apenas ao conhecimento de como criar endpoints ou consumir serviços, mas envolve uma visão aprofundada de como a plataforma gerencia recursos, executa fluxos internos e garante consistência durante a execução da aplicação. Este trabalho tem como objetivo apresentar, de forma estruturada e detalhada, os conceitos fundamentais relacionados ao ciclo de vida de uma API desenvolvida em .NET, abordando tanto o ciclo de vida da aplicação quanto o ciclo de vida das requisições.

2 Ciclo de vida da Aplicação

O ciclo de vida de uma aplicação se dá pelo processo de inicialização de uma API, incluindo todos os ciclos que ocorrem durante esse processo, o carregamento de configurações, a criação de host, o levantamento dos middlewares e também a injeção das dependências. Nesse capítulo serão descritos os processos que compõem o ciclo de vida da aplicação, com foco para o .NET.

O processo de inicialização de uma API desenvolvida em .NET é orquestrado pelo modelo de hospedagem (Hosting Model), introduzido de forma simplificada a partir do .NET 6 com o conceito de minimal hosting ([Microsoft, 2023](#)). Nesse modelo, o ponto de entrada da aplicação está concentrado no arquivo Program.cs, responsável por configurar e iniciar todos os componentes e configurações necessárias para que a API esteja apta a receber requisições HTTP ([Microsoft, 2023](#)) e indicar onde serão processadas.

Durante a inicialização, o primeiro passo consiste na criação do objeto responsável pela construção da aplicação, normalmente instanciado por meio do método *CreateBuilder(args)*. Esse builder centraliza a configuração dos serviços, do sistema de *logging*, do carregamento de configurações e do contêiner de injeção de dependência. Nesse momento, ocorre também a leitura das fontes de configuração da aplicação, como arquivos appsettings.json, appsettings.Environment.json, variáveis de ambiente e argumentos de linha de comando.

Após o registro dos serviços no contêiner de injeção de dependência, é realizada a construção da aplicação por meio do método *Build()*. Nesse estágio, o .NET gera internamente o ServiceProvider, responsável por gerenciar a criação e o ciclo de vida das dependências registradas ([Microsoft, 2025j](#)). A partir desse ponto, o conjunto de serviços passa a estar disponível para resolução durante a execução da aplicação.

O código a seguir exemplifica de forma prática como esse processo é feito. Esse código é referente à uma API que realmente está em uso, criada pela [SunSale System](#), ela é utilizada pelo projeto [Ordem](#).

```
1 using OrdemApi.Presentation.Api.App_Start;
2 using OrdemApi.Presentation.Api.Middleware;
3
4 var builder = WebApplication.CreateBuilder(args);
5
6 new Start(builder).Builder();
7
8 var app = builder.Build();
9
10 app.Use(async (context, next) =>
11 {
```

```
12  if (context.Request.Path == "/" )
13  {
14      context.Response.Redirect ("/swagger/index.html");
15  }
16  else
17  {
18      await next ();
19  }
20  });
21
22  app.UseCors(options =>
23      options.AllowAnyOrigin().AllowAnyMethod().AllowAnyHeader());
24
25  app.UseSwagger();
26  app.UseSwaggerUI(options =>
27  {
28      options.EnablePersistAuthorization();
29      options.DefaultModelExpandDepth(-1);
30      options.DefaultModelsExpandDepth(-1);
31      options.DocExpansion(Swashbuckle.AspNetCore.SwaggerUI.DocExpansion.None);
32      options.EnableFilter();
33  });
34
35  app.UseMiddleware<ErrorHandlingMiddleware>();
36
37  app.UseHttpsRedirection();
38
39  app.MapControllers();
40
41  app.UseAuthentication();
42  app.UseMiddleware<UserAccessMiddleware>();
43  app.UseAuthorization();
44
45  app.Run();
```

A partir desse código é possível ver de forma prática os passos criados para definição da inicialização da aplicação, tendo seu fim (ou realmente começo) no *app.Run()*.

Outro elemento fundamental nesse processo é a configuração do pipeline de requisições HTTP. O pipeline é composto por uma cadeia de middlewares, organizados de forma sequencial. Cada middleware é responsável por executar determinada lógica antes e/ou depois do processamento do próximo elemento da cadeia. Exemplos comuns incluem middlewares de tratamento global de exceções, autenticação, autorização, roteamento e compressão de resposta. A ordem em que esses middlewares são registrados é determinante para o comportamento da aplicação, pois define o fluxo de entrada e saída das requisições. O código a seguir, também do projeto [Ordem](#), é referente a um middleware de tratamento de exceções globais da aplicação.

```
1 namespace OrdemApi.Presentation.Api.Middleware
2 {
3     public class ErrorHandlingMiddleware
4     {
5         private readonly RequestDelegate _next;
6         private readonly IServiceScopeFactory _scopeFactory;
7
8         public ErrorHandlingMiddleware(RequestDelegate next,
9             IServiceScopeFactory scopeFactory)
10        {
11            _next = next;
12            _scopeFactory = scopeFactory;
13        }
14        public async Task Invoke(HttpContext context)
15        {
16            try
17            {
18                // Call the next middleware in the pipeline
19                await _next(context);
20            }
21            catch (Exception ex)
22            {
23                var errorResponse = new
24                {
25                    Message = "An error occurred",
26                    Exception = ex.Message,
27                    StackTrace = ex.StackTrace,
28                    Environment = Environment.GetEnvironmentVariable("ASPNETCORE_ENVIRONMENT")
29                };
30                // Serialize the error response to JSON
31                var json = JsonConvert.SerializeObject(errorResponse);
32                context.Response.ContentType = "application/json";
33                context.Response.StatusCode = 500;
34                using (var scope = _scopeFactory.CreateScope())
35                {
36                    var loggerAppService =
37                        scope.ServiceProvider.GetRequiredService<ILoggerAppService>();
38                    await loggerAppService.InsertAsync(ex);
39                }
40                // Write the JSON response to the body
41                await context.Response.WriteAsync(json);
42            }
43        }
44    }
45 }
```


Com esse código e com a injeção do Middleware no *Program.cs* é possível compreender como a execução é feita. Esse middleware funciona como um túnel, colocando todas as requisições no *try catch*, fazendo a chamada da requisição no próprio contexto, e logando qualquer erro que aconteça nessa chamada. A criação desse tipo de classe é importante para captura de erros, o que auxilia em *debugs* e manutenções pontuais de código ou banco.

Somente após a conclusão dessas etapas a aplicação inicia efetivamente sua execução por meio do método *Run()*, momento em que o servidor Kestrel passa a escutar a porta configurada e a aguardar conexões externas. A partir desse instante, o ciclo de vida da aplicação pode ser considerado ativo, permanecendo em execução contínua até que ocorra seu encerramento controlado ou falha inesperada.

A imagem a seguir ilustra o fluxo que acontece dentro da aplicação até que ela seja inicializada.

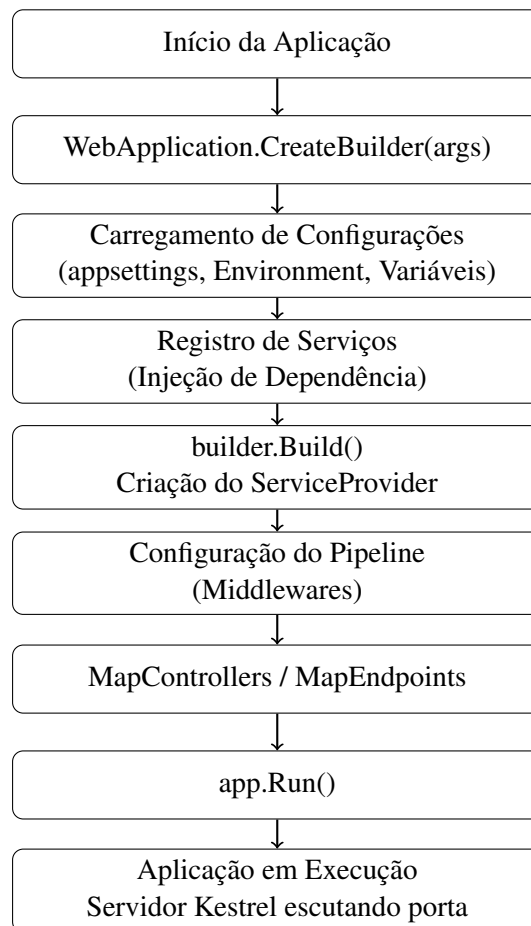


Figura 1 – Fluxo de inicialização de uma API .NET até o método *Run()*.

Embora o foco principal do ciclo de vida da aplicação esteja relacionado à sua inicialização, é igualmente importante compreender como ocorre seu encerramento. Em aplicações .NET, o ciclo de vida é gerenciado pelo Host, que controla tanto o início quanto o término da execução.

O encerramento da aplicação pode ocorrer de maneira controlada ou inesperada. No encerramento controlado, sinais do sistema operacional, como interrupções do processo (SIGTERM ou CTRL+C em ambiente de desenvolvimento), são capturados pelo Host. Nesse cenário, o .NET executa um processo de finalização ordenada (graceful shutdown), permitindo que requisições em andamento sejam concluídas antes da liberação dos recursos (Microsoft, 2025e). Durante esse processo, serviços registrados no contêiner de injeção de dependência que implementam interfaces como *IDisposable* ou *IAsyncDisposable* têm seus métodos de descarte invocados automaticamente.

Em ambientes de produção, especialmente quando executados em contêineres Docker ou orquestradores como Kubernetes, esse mecanismo é fundamental para garantir consistência e evitar perda de dados. O Host também pode ser configurado para executar ações específicas durante os eventos de inicialização e encerramento por meio de interfaces como *IHostedService* ou *BackgroundService*.

Por outro lado, encerramentos inesperados — como falhas críticas, exceções não tratadas no nível do Host ou falhas de infraestrutura — podem interromper abruptamente a execução da aplicação, impedindo o descarte adequado de recursos. Esse tipo de cenário reforça a importância de boas práticas, como tratamento global de exceções e monitoramento contínuo da aplicação.

Dessa forma, o ciclo de vida da aplicação .NET não se limita apenas ao processo de inicialização até o método *Run()*, mas compreende também o gerenciamento adequado de seu encerramento. Esse comportamento estruturado reforça a robustez do modelo de hospedagem da plataforma e prepara o terreno para a compreensão do funcionamento interno da injeção de dependência, tema abordado no próximo capítulo.

2.1 Síntese do capítulo

O ciclo de vida da aplicação .NET envolve um conjunto de etapas que ocorrem desde a inicialização do host até o encerramento controlado da execução da API. Durante esse processo são carregadas as configurações da aplicação, registrados os serviços no contêiner de injeção de dependência e configurado o pipeline de middlewares que será responsável por processar as requisições HTTP.

Compreender esse fluxo é fundamental para entender como a aplicação é estruturada internamente e como o framework gerencia seus componentes durante a execução. Esse conhecimento também permite identificar pontos estratégicos para configuração de serviços, definição de middlewares e tratamento global de exceções.

Entre os elementos que surgem nesse processo, destaca-se o contêiner de injeção de dependência, responsável por gerenciar a criação e o ciclo de vida das dependências utilizadas

pela aplicação. Dessa forma, o entendimento do ciclo de vida da aplicação serve como base para compreender o funcionamento da injeção de dependência no ecossistema .NET, tema abordado no próximo capítulo.

3 Injeção de Dependência no .NET

A injeção de dependência (Dependency Injection – DI) é um dos pilares fundamentais da arquitetura moderna no ecossistema .NET. Seu objetivo principal é promover o desacoplamento entre classes, permitindo que dependências sejam fornecidas externamente, em vez de criadas diretamente dentro dos componentes (MACHADO, 2025).

Em aplicações tradicionais sem o uso de DI, uma classe normalmente instancia suas próprias dependências, o que gera forte acoplamento e dificulta testes, manutenção e evolução do sistema. A injeção de dependência resolve esse problema ao delegar a responsabilidade de criação e gerenciamento dos objetos a um contêiner especializado (MACHADO, 2025).

No .NET, esse mecanismo está integrado nativamente ao framework, sendo configurado durante o ciclo de vida da aplicação, especialmente no momento da construção do host, onde é colocado o mapeamento para permitir essa injeção. A partir da criação do *ServiceProvider*, o contêiner passa a ser responsável por gerenciar a criação, reutilização e descarte das dependências registradas nesse *provider* (Microsoft, 2025n).

Dessa forma, compreender o funcionamento da injeção de dependência é essencial para entender tanto a estrutura interna da aplicação quanto o comportamento das requisições em tempo de execução.

3.1 Conceito e Contêiner de Serviços (*Service Container*)

A injeção de dependência é um padrão de projeto que implementa o princípio da Inversão de Controle (*Inversion of Control – IoC*). A Inversão de Controle ocorre quando a responsabilidade pela criação de objetos deixa de ser da própria classe consumidora e passa a ser delegada a uma entidade externa. Nesse sentido, a injeção de dependências resolve problemas de dependência codificados por meio da utilização de interfaces ou classes bases para abstrair a implementação da dependência, registro da dependência em um contêiner de serviços e a injeção do serviço no construtor da classe em que ele é usado (Microsoft, 2025n).

No contexto do .NET, essa entidade externa é o contêiner de serviços (*Service Container*), responsável por manter um registro de tipos e suas respectivas implementações.

Exemplo conceitual:

Sem DI:

```
1 public class PedidoService
2 {
3     private readonly PedidoRepository _repository = new PedidoRepository();
4 }
```

Com DI:

```
1 public class PedidoService
2 {
3     private readonly IPedidoRepository _repository;
4     public PedidoService(IPedidoRepository repository)
5     {
6         _repository = repository;
7     }
8 }
```

A classe não cria mais sua dependência, ela apenas declara que precisa dela. O controlador de dependências então é o grande responsável por definir qual o objeto correto para a interface que está explícita no construtor da classe. Nota-se que ao introduzir esse conceito você entrega o de inversão de dependências, que também está explícito nos padrões *SOLID*, deixando o código mais limpo e desacoplado (MACHADO, 2025).

A configuração da injeção de dependência ocorre durante a inicialização da aplicação, no momento em que os serviços são registrados no objeto *builder.Services*. Esse registro informa ao contêiner como instanciar e gerenciar cada tipo. Exemplo:

```
1 builder.Services.AddTransient<IPedidoRepository, PedidoRepository>();
```

Após a execução do método *Build()*, o .NET cria o *ServiceProvider*, que passa a ser responsável pela resolução das dependências durante toda a execução da aplicação.

3.2 Ciclo de Vida das Dependências

Com a injeção de dependências é introduzido também o conceito de ciclo de vida dessas dependências, que está diretamente associado com o ciclo de vida de requisições e da aplicação. Ao introduzir um laço entre uma interface e uma classe, é necessário entender como o objeto é instanciado e como essa instância funciona de acordo com a necessidade das classes que a utiliza (Microsoft, 2025k). Além disso, o contêiner do .NET não apenas cria as instâncias, mas também controla o descarte automático de objetos que implementam *IDisposable* ou *IAsyncDisposable*, respeitando seu ciclo de vida configurado.

No .NET existem três tempos de vidas principais para a injeção de dependências:

- *Singleton* – Uma única instância para toda a aplicação. Criada uma vez e reutilizada até o encerramento do host.
- *Scoped* – Uma instância por escopo. Em aplicações web, o escopo normalmente corresponde a uma requisição HTTP.

- *Transient* – Uma nova instância a cada resolução. Toda vez que a dependência é identificada, uma nova instância é criada, sendo descartada ao final de sua utilização.

Com o ciclo de vida de cada injeção de dependência possuindo suas próprias particularidades, é necessário ter um entendimento de cada um para que, no momento de escolha de qual usar, se consiga aplicar aquele que mais faz sentido para seu cenário (Microsoft, 2025k). Abaixo será listado e exemplificado cada um deles.

3.2.1 *Singleton*

O tempo de vida Singleton representa dependências que são instanciadas apenas uma única vez durante todo o ciclo de vida da aplicação. Após sua criação, a mesma instância é reutilizada em todas as resoluções feitas pelo contêiner de injeção de dependência até que a aplicação seja encerrada (Refactoring.Guru, 2025). A imagem abaixo apresenta isso de forma ilustrativa.

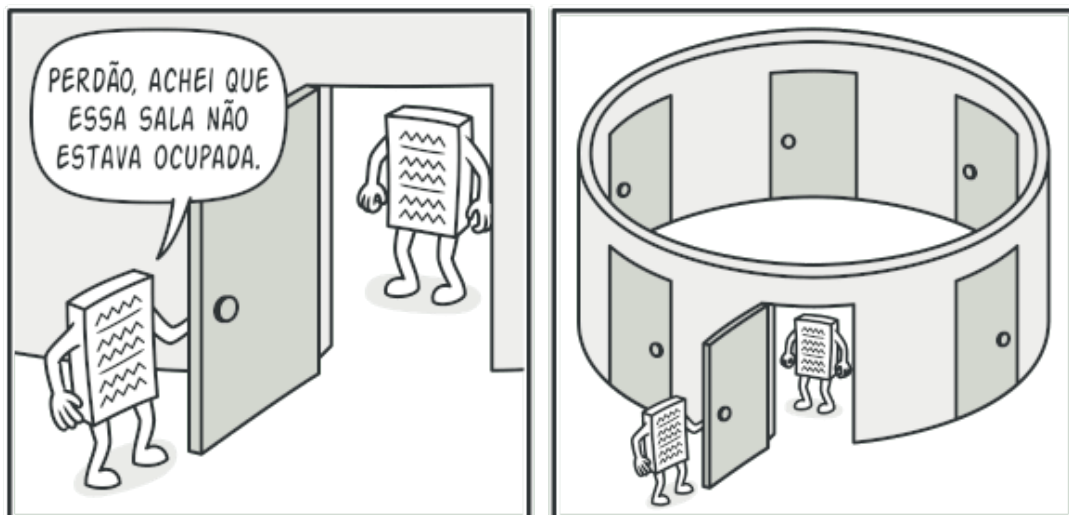


Figura 2 – *Singleton* - Clientes podem não se dar conta que estão lidando com o mesmo objeto a todo momento (Refactoring.Guru, 2025).

Pela imagem é possível identificar que um mesmo objeto é usado em todas as chamadas, independentemente de onde ela seja feita, uma vez que o objeto foi instanciado uma única vez e está diretamente ligado ao ciclo de vida da aplicação.

Esse tipo de registro é indicado para serviços que não possuem estado dependente de requisições e que podem ser compartilhados com segurança entre diferentes partes da aplicação. Como a instância é única, qualquer alteração em seu estado será visível para todos os consumidores dessa dependência.

No .NET, serviços Singleton são registrados por meio do método *AddSingleton*. Uma vez criado, o objeto permanece em memória até o encerramento da aplicação, momento em

que o contêiner executa automaticamente seu descarte caso a classe implemente as interfaces *IDisposable* ou *IAsyncDisposable*.

Um exemplo comum de utilização desse tempo de vida é em serviços de configuração ou caches compartilhados pela aplicação.

```
1 builder.Services.AddSingleton<IConfigurationService,
    ConfigurationService>();
```

Nesse exemplo, a instância de *ConfigurationService* será criada apenas uma vez durante toda a execução da aplicação e reutilizada sempre que for solicitada por qualquer componente (NIZZOLA, 2023). Então se em uma requisição de uma aplicação eu estou capturando uma variável de ambiente que está acoplada a essa *ConfigurationService*, ela usará a mesma instância de uma outra requisição, que também está utilizando essa mesma instância do objeto.

Ao utilizar o *Singleton* deve-se ter em mente que está sendo sempre utilizada uma mesma instância e, em casos onde são alteradas propriedades ou mesmo recriada uma nova instância para o objeto, isso valerá para todos os clientes que estão consumindo essa dependência. Devem-se entender os cenários para escolher se esse registro é o mais indicado, mas comumente ele é utilizado para *logs*, acesso a variáveis de ambiente e para acesso a arquivos estáticos (NIZZOLA, 2023).

3.2.2 Scoped

O tempo de vida *Scoped* representa dependências cuja instância é criada uma vez para cada escopo de execução. Em aplicações web desenvolvidas com ASP.NET Core, esse escopo normalmente corresponde ao ciclo de vida de uma requisição HTTP, ou uma requisição dentro de uma API. Fora do contexto web, é possível criar escopos manualmente utilizando o *IServiceScopeFactory*. Ele é indicado para serviços que precisam de estado por solicitação, como repositórios de banco de dados que gerenciam transações, por exemplo (NIZZOLA, 2023).

Isso significa que, durante o processamento de uma requisição, todas as resoluções daquele serviço receberão a mesma instância. No entanto, para uma nova requisição, mesmo que seja para a mesma rota, uma nova instância será criada.

Esse comportamento é particularmente útil para serviços que precisam manter consistência durante o processamento de uma requisição, mas que não devem compartilhar estado entre requisições diferentes. Dessa forma, o compartilhamento se dá pelo escopo de onde a dependência está sendo utilizada, não por toda a aplicação, como é feito no *Singleton*.

No .NET, serviços *Scoped* são registrados utilizando o método *AddScoped*.

```
1 builder.Services.AddScoped<IOrderService, OrderService>();
```

Durante o processamento de uma requisição HTTP, todas as classes que dependam de *IOrderService* receberão a mesma instância de *OrderService*. Entretanto, quando uma nova

requisição for recebida pela aplicação, uma nova instância será criada pelo contêiner. Dessa forma o compartilhamento da instância do objeto não se dá em chamadas diferentes, garantindo estado único, mesmo quando se tem concorrência.

Um exemplo clássico de utilização desse tempo de vida é o *DbContext* do Entity Framework, que precisa manter consistência durante toda a operação da requisição, mas não deve ser compartilhado entre diferentes usuários ou chamadas simultâneas. Quando aplicado o escopo para a instância do banco de dados, cada cliente utilizará sua conexão com o banco, enquanto que com solicitações diferentes, cada uma terá o seu acesso (NIZZOLA, 2023).

3.2.3 *Transient*

O tempo de vida *Transient* representa dependências que são instanciadas sempre que são solicitadas ao contêiner de serviços. Diferentemente dos tempos de vida *Singleton* e *Scoped*, não há reutilização da instância. Isso significa que, sempre que uma classe solicitar essa dependência, uma nova instância será criada pelo contêiner de injeção de dependência e, ao final da utilização, a instância é descartada (Microsoft, 2025k).

Esse tipo de registro é recomendado para serviços leves, sem estado interno ou que executam operações pontuais, onde não há necessidade de reutilização da instância. Ele, dentro os demais, é o mais custoso, uma vez que a cada solicitação de um objeto ele cria a instância do mesmo, descartando quando o mesmo não é utilizado mais (Microsoft, 2025k).

No .NET, serviços *Transient* são registrados por meio do método *AddTransient*.

```
1 builder.Services.AddTransient<ILogFormatter, LogFormatter>();
```

Nesse caso, sempre que a interface *ILogFormatter* for solicitada, uma nova instância da classe *LogFormatter* será criada pelo contêiner, e sempre que a mesma cair em desuso, a instância é perdida.

Embora esse modelo seja simples, é importante utilizá-lo com cautela em serviços mais complexos ou que possuam custo elevado de criação, pois múltiplas instâncias podem ser geradas durante o processamento de uma única requisição, principalmente quando a criação de uma nova instância pode ser mais custosa, com acesso a banco e levantamento de serviços.

3.3 *Disposal of Services*

O contêiner de injeção de dependência do .NET não é responsável apenas pela criação das dependências utilizadas pela aplicação, mas também pelo gerenciamento adequado do ciclo de vida desses objetos. Isso inclui o descarte automático de serviços que implementam as interfaces *IDisposable* ou *IAsyncDisposable* (Microsoft, 2025d).

Quando um serviço registrado no contêiner implementa uma dessas interfaces, o próprio framework garante que o método *Dispose()* ou *DisposeAsync()* seja executado no momento apropriado, liberando recursos utilizados pela instância (Microsoft, 2025d).

O momento em que o descarte ocorre depende diretamente do tempo de vida configurado para a dependência. Serviços registrados como *Singleton* são descartados apenas quando a aplicação é encerrada. Já serviços registrados como *Scoped* são descartados ao final do escopo da requisição HTTP, enquanto serviços *Transient* são descartados após o término de sua utilização dentro do escopo em que foram criados.

Esse mecanismo é particularmente importante em serviços que utilizam recursos externos, como conexões com banco de dados, arquivos ou serviços de rede. O descarte correto desses recursos evita vazamentos de memória e garante a estabilidade da aplicação em ambientes com grande volume de requisições.

Um exemplo comum desse comportamento pode ser observado no *DbContext* do Entity Framework, que implementa a interface *IDisposable*. Quando registrado como *Scoped*, o contêiner garante que sua instância seja automaticamente descartada ao final da requisição HTTP.

3.4 Síntese do capítulo

Neste capítulo foram apresentados os principais conceitos relacionados à injeção de dependência no ecossistema .NET, com foco no funcionamento do contêiner de serviços e nos diferentes tempos de vida das dependências.

Dessa forma, a compreensão dos diferentes tempos de vida das dependências é essencial para o desenvolvimento de aplicações robustas no ecossistema .NET. O contêiner de injeção de dependência não apenas facilita o desacoplamento entre componentes, mas também assume a responsabilidade pelo gerenciamento do ciclo de vida dos objetos utilizados pela aplicação.

A escolha adequada entre os tempos de vida *Singleton*, *Scoped* e *Transient* deve considerar o comportamento esperado do serviço, o escopo de sua utilização e o impacto que o compartilhamento ou a recriação de instâncias pode causar na consistência e no desempenho da aplicação. Além disso, o gerenciamento automático de descarte realizado pelo contêiner contribui para a liberação adequada de recursos, evitando problemas como vazamento de memória e conexões não finalizadas.

Compreender esses mecanismos é fundamental para projetar aplicações escaláveis, manuteníveis e alinhadas às boas práticas da plataforma .NET. Esse entendimento também prepara o terreno para a análise do ciclo de vida de uma requisição HTTP, tema abordado no próximo capítulo.

4 Ciclo de Vida da Requisição

Após compreender o processo de inicialização da aplicação e o funcionamento da injeção de dependência no ecossistema .NET, torna-se necessário analisar como as requisições são efetivamente processadas durante a execução da API. Enquanto o ciclo de vida da aplicação está relacionado à configuração e inicialização da infraestrutura necessária para o funcionamento do sistema, o ciclo de vida da requisição descreve o fluxo percorrido por cada chamada HTTP realizada pelos clientes ([Microsoft Tech Community, 2024](#)).

Em aplicações desenvolvidas com ASP.NET Core, cada requisição passa por uma série de etapas internas que envolvem o recebimento da chamada pelo servidor, o processamento por meio da execução dos middlewares, a resolução de dependências pelo contêiner de serviços e a execução do endpoint responsável pela lógica da operação solicitada ([Microsoft Tech Community, 2024](#)). Durante esse processo, mecanismos importantes, como a execução assíncrona, gerenciamento de escopos de dependência e descarte automático de recursos, são utilizados pelo framework para garantir eficiência e escalabilidade.

Para demonstrar de forma prática o fluxo de processamento de uma requisição em uma API .NET, será utilizado como exemplo o projeto Ordem, desenvolvido pela SunSale System. Essa API será acessada por meio da ferramenta Postman, permitindo observar as etapas envolvidas desde o envio da requisição até o retorno da resposta.

O projeto possui uma estrutura baseada em controllers e utiliza middlewares para tratamento de exceções e controle de acesso. Esses componentes já foram apresentados anteriormente neste trabalho e serão utilizados novamente para ilustrar como a requisição percorre o pipeline da aplicação.

Este capítulo apresenta de forma detalhada as etapas que compõem o ciclo de vida de uma requisição em uma API .NET, descrevendo os principais componentes envolvidos nesse processo e como eles interagem durante o processamento de uma chamada HTTP. Nele é também ilustrado de forma objetiva como a requisição chega do lado da API e como ela é retornada para o cliente responsável pela solicitação.

4.1 Conceito de requisição em aplicações web

Em aplicações web baseadas em APIs, a comunicação entre cliente e servidor ocorre por meio de requisições HTTP. Uma requisição representa uma chamada realizada por um cliente, como um navegador, aplicação mobile ou outro serviço, solicitando a execução de uma determinada operação exposta por uma API, ou mesmo a chamada de um recurso web, como um arquivo estático ([Codecademy, 2024](#)).

Cada requisição HTTP contém informações importantes que permitem ao servidor compreender qual operação deve ser executada. Entre essas informações estão o método HTTP utilizado (como *GET*, *POST*, *PUT* ou *DELETE*, conhecido como verbo HTTP), o endereço do recurso solicitado, os cabeçalhos da requisição e, quando necessário, um corpo contendo dados enviados pelo cliente ([Codecademy, 2024](#)).

No ecossistema do ASP.NET Core, quando uma requisição chega ao servidor, ela é recebida inicialmente pelo servidor web utilizado pela aplicação, normalmente o *Kestrel*. A partir desse momento, o framework cria uma estrutura chamada *HttpContext*, responsável por representar todo o contexto daquela chamada.

O objeto *HttpContext* contém diversas informações relacionadas à requisição e à resposta, incluindo os dados enviados pelo cliente, os cabeçalhos HTTP, os parâmetros de rota e também a estrutura que permitirá a construção da resposta que será retornada ([Microsoft Tech Community, 2024](#)). Esse objeto acompanha todo o ciclo de processamento da requisição dentro da aplicação.

A partir da criação do *HttpContext*, a requisição é encaminhada para o pipeline de processamento do ASP.NET Core, composto por uma sequência de middlewares responsáveis por executar diferentes etapas da lógica da aplicação. Esse fluxo continuará até que o endpoint responsável pela operação solicitada seja identificado e executado.

4.2 Recebimento da requisição pelo servidor

Quando a aplicação .NET está em execução, após a chamada do método *Run()*, o servidor web configurado, normalmente o *Kestrel*, passa a escutar a porta definida e aguardar requisições HTTP provenientes de clientes. O serviço fica então apto para receber comunicação externa de clientes, processando essas informações conforme configurado antes do comando *Run()* da aplicação.

Os comandos executados até o início de fato da aplicação são as configurações aplicadas à mesma. Essas configurações, já mencionadas anteriormente, são basicamente para expor as rotas e os métodos que serão disponibilizados pelo serviço, assim como aplicar *middlewares* e serviços, como a própria injeção de dependência.

Ao utilizar ferramentas como o *Postman* ([Postman, Inc., 2025](#)), é possível simular esse comportamento enviando requisições diretamente para os endpoints expostos pela API, assim como um cliente consumindo o serviço. No contexto deste trabalho, será utilizado o projeto *Ordem* ([MACHADO, 2026](#)) para exemplificar esse fluxo, permitindo observar de forma prática como a requisição é recebida e processada.

Quando uma requisição HTTP é enviada para a aplicação, o servidor *Kestrel* é responsável por receber essa conexão e iniciar o processamento dentro do ASP.NET Core ([Microsoft,](#)

2025g). A partir desse momento, o framework cria um objeto chamado *HttpContext*, que representa todo o contexto da requisição. Na aplicação *Ordem*, esse processo é iniciado com o seguinte trecho de código:

```
1 using OrdemApi.Presentation.Api.App_Start;
2 using OrdemApi.Presentation.Api.Middleware;
3
4 // O metodo CreateBuilder(args) configura automaticamente o servidor
   Kestrel como servidor padrao da aplicacao.
5 var builder = WebApplication.CreateBuilder(args);
6
7 new Start(builder).Builder();
```

Ao executar essa configuração, a aplicação passa a utilizar o servidor *Kestrel*, sendo possível lidar com um alto volume de conexões de forma eficiente e fornecendo segurança contra várias vulnerabilidades de servidores web (inclusive já oferecendo suporte à *HTTPS*). Ademais, ele possibilita vários tipos diferentes de projetos no mundo *.NET*, como *APIs*, *MVC*, *Razor pages*, *SignalR*, *Blazor*, e *gRPC* (Microsoft, 2025g).

O *HttpContext* contém informações essenciais encapsuladas, como os dados da requisição (*HttpRequest*), a estrutura da resposta (*HttpResponse*), os cabeçalhos HTTP, parâmetros de rota, informações de autenticação e outros dados relevantes (Microsoft, 2025m). Esse objeto acompanha toda a execução da requisição dentro da aplicação.

Após a criação do *HttpContext*, a requisição é encaminhada para o pipeline de processamento do ASP.NET Core, onde passará por uma sequência de middlewares configurados durante a inicialização da aplicação (Microsoft, 2025m). Cada um desses componentes terá a oportunidade de atuar sobre a requisição antes que ela chegue ao endpoint responsável pela execução da lógica solicitada.

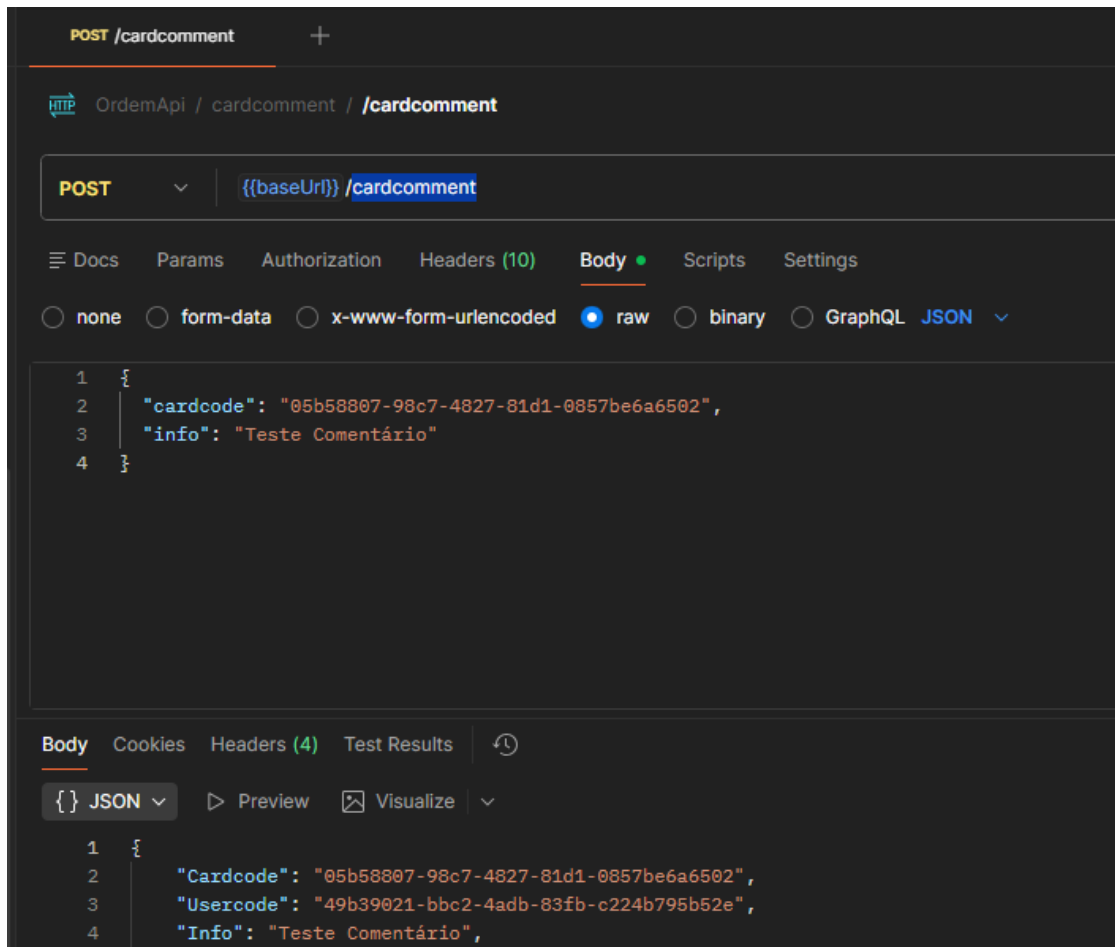


Figura 3 – Envio de requisição HTTP ao endpoint *cardcomment* da API Ordem utilizando o método *POST*.

Na Figura 3, é possível observar o envio de uma requisição HTTP para um dos *endpoints* da API Ordem (*endpoint cardcomment*), que será utilizada como base para exemplificar o fluxo de processamento ao longo deste capítulo.

A partir desse ponto, a requisição passa a percorrer o pipeline de middlewares da aplicação, onde cada componente poderá atuar sobre o fluxo antes que o endpoint final seja executado. Esse processo será detalhado na próxima seção.

4.3 Pipeline de middlewares

O pipeline de middlewares é um dos principais mecanismos responsáveis pelo processamento de requisições no *ASP.NET Core*. Ele consiste em uma cadeia de componentes organizados de forma sequencial, onde cada middleware possui a responsabilidade de executar determinada lógica durante o processamento da requisição ([Microsoft, 2025h](#)).

Cada middleware pode atuar em dois momentos distintos: antes da execução do próximo componente da cadeia e após o retorno da execução. Esse comportamento permite que os middlewares funcionem como um fluxo bidirecional, onde a requisição percorre o pipeline até o endpoint e, posteriormente, a resposta percorre o caminho inverso ([Microsoft, 2025h](#)).

A configuração do pipeline é realizada durante a inicialização da aplicação, no arquivo *Program.cs*, por meio da utilização de métodos como *Use*, *UseMiddleware*, *UseAuthentication* e *UseAuthorization*. A ordem em que esses middlewares são registrados é fundamental, pois define diretamente o fluxo de execução da requisição ([Microsoft, 2025h](#)). Tomando como exemplo o projeto *Ordem*, é possível observar no código a seguir como os middlewares são registrados na aplicação:

```
1
2 // Adicionando middleware para tratativa de erros e excecoes
3 app.UseMiddleware<ErrorHandlingMiddleware>();
4
5 // Adicionando middleware para validacao de acesso de usuarios
6 app.UseMiddleware<UserAccessMiddleware>();
7
8 app.UseHttpsRedirection();
9 app.MapControllers();
10 app.UseAuthentication();
11 app.UseAuthorization();
12
13 app.Run();
```

Como demonstrado no código anterior, é possível observar a configuração de dois middlewares responsáveis por diferentes aspectos do processamento da requisição, um responsável pelo tratamento de exceções e outro dedicado ao controle de acesso da aplicação, além dos middlewares nativos do framework responsáveis por autenticação e autorização ([Microsoft, 2025a](#)). A seguir, é apresentado o código da classe *ErrorHandlingMiddleware*, já apresentado anteriormente na introdução desse trabalho:

```
1 public class ErrorHandlerMiddleware
2 {
3     private readonly RequestDelegate _next;
4     private readonly IServiceScopeFactory _scopeFactory;
5
6     public ErrorHandlerMiddleware(RequestDelegate next,
7                                 IServiceScopeFactory scopeFactory)
8     {
9         _next = next;
10        _scopeFactory = scopeFactory;
11    }
12    public async Task Invoke(HttpContext context)
13    {
14        try
15        {
16            await _next(context);
17        }
18        catch (Exception ex)
19        {
20            var errorResponse = new
21            {
22                Message = "An error occurred",
23                Exception = ex.Message,
24                ex.StackTrace,
25                Environment =
26                    Environment.GetEnvironmentVariable("ASPNETCORE_ENVIRONMENT")
27            };
28            var json = JsonConvert.SerializeObject(errorResponse);
29            context.Response.ContentType = "application/json";
30            context.Response.StatusCode = 500;
31            using (var scope = _scopeFactory.CreateScope())
32            {
33                var loggerAppService =
34                    scope.ServiceProvider.GetRequiredService<ILoggerAppService>();
35                await loggerAppService.InsertAsync(ex);
36            }
37            await context.Response.WriteAsync(json);
38        }
39    }
40 }
```

O código apresenta o construtor responsável por receber as dependências do middleware, bem como o método *Invoke*, que contém a lógica de execução durante o processamento da requisição.

Ao receber uma requisição, o *ASP.NET Core* inicia sua execução a partir do primeiro

middleware registrado no pipeline. Cada *middleware* possui acesso ao contexto da requisição e a um *delegate* que representa o próximo componente da cadeia ([Microsoft, 2025a](#)).

Esse *delegate* é normalmente representado pelo parâmetro *next*, visto no código apresentado anteriormente, que permite a continuidade do fluxo. Ao invocar *await next(context)*, o *middleware* transfere o controle para o próximo elemento do pipeline. Caso essa chamada não seja realizada, a execução da requisição é interrompida naquele ponto ([Microsoft, 2025a](#)).

Esse modelo de execução permite que cada *middleware* atue como um ponto de interceptação no fluxo da requisição, podendo modificar tanto os dados de entrada quanto a resposta final. Esse comportamento torna o pipeline altamente flexível e extensível, permitindo a implementação de funcionalidades transversais, como *logging*, tratamento de erros (como é o caso do nosso *middleware*) e segurança.

Dessa forma, o pipeline de *middlewares* pode ser entendido como o núcleo do processamento de requisições no ASP.NET Core, sendo responsável por orquestrar todas as etapas que ocorrem entre o recebimento da requisição e a execução do endpoint.

Após percorrer o pipeline de *middlewares*, a requisição chega ao mecanismo de roteamento da aplicação, onde o endpoint responsável pela operação solicitada é identificado e executado ([Microsoft, 2025a](#)). Nesse momento, ocorre também a resolução das dependências necessárias para a execução da lógica da aplicação, conforme será apresentado na próxima seção.

4.4 Resolução de dependências durante a requisição

Durante o processamento de uma requisição no ASP.NET Core, um dos mecanismos fundamentais que entram em ação é a resolução de dependências por meio do contêiner de injeção de dependência ([Microsoft, 2025n](#)). Conforme apresentado no capítulo anterior, o contêiner é responsável por gerenciar a criação e o ciclo de vida dos serviços utilizados pela aplicação.

No contexto do ciclo de vida da requisição, o ASP.NET Core cria automaticamente um escopo de execução para cada requisição HTTP recebida. Esse escopo define o contexto no qual as dependências serão resolvidas, garantindo isolamento entre diferentes requisições ([Microsoft, 2025n](#)).

Dentro desse escopo, todas as dependências registradas com tempo de vida *Scoped* são instanciadas uma única vez e compartilhadas durante toda a execução da requisição. Já dependências registradas como *Transient* são criadas a cada solicitação, enquanto dependências *Singleton* permanecem compartilhadas entre todas as requisições da aplicação ([Microsoft, 2025n](#)).

Esse comportamento pode ser observado durante a execução de um endpoint da API *Ordem*, onde serviços são automaticamente injetados nos controllers por meio do construtor. Ao receber uma requisição, o ASP.NET Core utiliza o contêiner para resolver todas as dependências

necessárias antes da execução do método responsável pela operação solicitada.

Ainda com o exemplo demonstrado na figura 3, aqui está parte da controller onde a requisição é processada:

```
1 [ApiController]
2 [Authorize]
3 [Route("cardcomment")]
4 public class CardcommentController : ControllerBase, IDisposable
5 {
6     private readonly IMainAppService _mainAppService;
7     private readonly ICardAppService _cardAppService;
8     private readonly ICardloggerAppService _cardloggerAppService;
9     private readonly IOfficeusersAppService _officeusersAppService;
10    private readonly Settings _settings;
11
12    private readonly TokenHandler tokenController;
13
14    /// <summary>
15    /// Class constructor
16    /// </summary>
17    public CardcommentController(IMainAppService mainAppService,
18                                ICardAppService cardAppService, ICardloggerAppService
19                                cardloggerAppService, IOfficeusersAppService officeusersAppService,
20                                IOption<Settings> options, IHttpContextAccessor httpContextAccessor)
21    {
22        _mainAppService = mainAppService;
23        _cardAppService = cardAppService;
24        _cardloggerAppService = cardloggerAppService;
25        _officeusersAppService = officeusersAppService;
26        _settings = options.Value;
27
28        tokenController = new TokenHandler(httpContextAccessor, options);
29    }
30
31    [HttpPost]
32    public async Task<IActionResult> Post([FromBody] MainViewModel model)
33    {
34        var mainDto = model.ProjectedAs<MainDTO>();
35        var card = await _cardAppService.GetAsync(mainDto.Cardcode);
36        var userInfo = await tokenController.GetUserFromRequest();
37        if (card == null || !await CanAccessOffice(userInfo, card.Officecode))
38        {
39            return Forbid();
40        }
41        if (userInfo?.code != Guid.Empty)
42        {
43            mainDto.Usercode = userInfo.code;
44        }
45    }
46}
```

```

41     }
42     var result = await _mainAppService.InsertAsync(mainDto);
43
44     if (userInfo?.code != Guid.Empty)
45     {
46         await _cardloggerAppService.InsertAsync(new CardloggerDTO
47         {
48             Cardcode = mainDto.Cardcode,
49             Usercode = userInfo.code,
50             Info = "Comentário adicionado"
51         });
52     }
53
54     return Ok(result);
55 }
56 }

```

No construtor é possível ver a listagem de dependências da classe. Dessa forma, no momento em que o controller é instanciado pelo framework, suas dependências são automaticamente resolvidas pelo contêiner de injeção de dependência conforme configuradas pelo *Builder*:

```

1  // program.cs
2  var builder = WebApplication.CreateBuilder(args);
3
4  new Start(builder).Builder();
5
6  //start.Builder
7  DependencyResolverConfig.Register(this._app);
8
9  // DependencyResolverConfigRegister
10 public static void Register(WebApplicationBuilder app)
11 {
12     _ = new DependencyResolverContainer(app.Services);
13 }
14
15 //DependencyResolverContainer
16 public DependencyResolverContainer(IServiceCollection services)
17 {
18     RegisterServices(services);
19 }
20
21 public void RegisterService<TService,
22     TImplementation>(IServiceCollection services)
23     where TService : class
24     where TImplementation : class, TService
25 {
26     services.AddScoped<TService, TImplementation>();
27 }

```

```
26     }
27
28     public void RegisterServices(IServiceCollection services)
29     {
30         RegisterService<ICardcommentAppService,
31             CardcommentAppService>(services);
32     }
```

No código acima é possível identificar como a referência da *ICardcommentAppService* é aplicada. Dessa forma, o contêiner de injeção de dependência passa a conhecer qual implementação concreta deve ser utilizada sempre que a interface *ICardcommentAppService* for solicitada, nesse caso, a classe *CardcommentAppService*.

Esse processo de resolução ocorre de forma recursiva, ou seja, caso uma dependência possua outras dependências em seu construtor, o contêiner também será responsável por resolvê-las automaticamente, formando uma cadeia de resolução até que todos os objetos necessários estejam disponíveis para execução.

Pode-se notar ainda que, embora seja possível implementar a interface *IDisposable* diretamente em controllers, na maioria dos cenários o descarte de dependências é gerenciado automaticamente pelo contêiner de injeção de dependência. Então no código acima da *controller*, onde é citada a interface *IDisposable*, não seria necessário.

Ao final do processamento da requisição, o escopo criado pelo ASP.NET Core é encerrado automaticamente. Nesse momento, o contêiner executa o descarte das dependências que implementam as interfaces *IDisposable* ou *IAsyncDisposable*, respeitando o tempo de vida definido para cada serviço.

Esse comportamento garante que recursos como conexões com banco de dados e objetos de acesso externo sejam liberados corretamente, evitando vazamentos de memória e problemas de desempenho. Dessa forma, a resolução de dependências durante a requisição garante que cada chamada HTTP seja processada de forma isolada, mantendo consistência interna e assegurando o gerenciamento adequado dos recursos utilizados pela aplicação (Microsoft, 2025d).

Durante esse processo, é comum que os serviços executem operações assíncronas, como consultas a banco de dados ou chamadas a serviços externos. Esse modelo de execução será detalhado na próxima seção, abordando o uso de métodos assíncronos no ASP.NET Core.

4.5 Execução assíncrona no ASP.NET Core

Durante o processamento de uma requisição em aplicações desenvolvidas com ASP.NET Core, é comum que determinadas operações envolvam acesso a recursos externos, como consultas a banco de dados, chamadas a serviços HTTP ou leitura de arquivos. Essas operações, por natureza, podem levar um tempo considerável para serem concluídas (Microsoft, 2025i).

Para lidar com esse tipo de cenário de forma eficiente, o .NET utiliza o modelo de execução assíncrona baseado nas palavras-chave *async* e *await*. Esse modelo permite que a aplicação continue processando outras requisições enquanto aguarda a conclusão de operações externas, melhorando significativamente a utilização dos recursos do servidor (Microsoft, 2025i).

No ASP.NET Core, o processamento de requisições é realizado por meio de um conjunto de threads gerenciadas pelo *Thread Pool* (Microsoft, 2025l). O *Thread Pool* fornece à aplicação um conjunto de *threads* gerenciadas pelo sistema operacional de trabalho, permitindo que o desenvolvimento se concentre nas tarefas da aplicação, ao invés de gerenciar as *threads*. Se a aplicação possuir tarefas curtas que exijam processamento em segundo plano, o *pool* de *threads* gerenciados será uma maneira fácil de tirar proveito de várias *threads* (Microsoft, 2025l). Quando uma requisição é recebida, uma *thread* disponível no *pool* é utilizada para iniciar sua execução.

Ao encontrar uma operação assíncrona, como uma chamada ao banco de dados utilizando *await*, a *thread* atual não permanece bloqueada aguardando a resposta. Em vez disso, ela é liberada para atender outras requisições, enquanto o runtime aguarda a conclusão da operação externa (Microsoft, 2025l). Esse modelo permite que a aplicação maximize a utilização das *threads* disponíveis, evitando bloqueios desnecessários e aumentando a capacidade de processamento concorrente.

Quando a operação é finalizada, a continuação da execução é agendada em uma *thread* disponível no *Thread Pool*, permitindo que o processamento da requisição seja retomado a partir do ponto onde foi interrompido.

Esse comportamento pode ser observado no método *Post* da controller apresentada anteriormente, onde diversas chamadas assíncronas são realizadas:

```
1 var card = await _cardAppService.GetAsync(mainDto.Cardcode);  
2 var userInfo = await tokenController.GetUserFromRequest();  
3 var result = await _mainAppService.InsertAsync(mainDto);
```

Em cada uma dessas chamadas, a execução da requisição é temporariamente pausada até que a operação seja concluída. Durante esse período, a *thread* utilizada para processar a requisição é liberada, permitindo que o servidor utilize seus recursos de forma mais eficiente.

Esse modelo é especialmente importante em aplicações web com alto volume de acessos, pois evita o bloqueio desnecessário de threads, permitindo que múltiplas requisições sejam processadas de forma concorrente.

É importante destacar que o modelo assíncrono é mais eficiente em operações do tipo *I/O-bound*, como acesso a banco de dados ou chamadas externas. Em operações do tipo *CPU-bound*, que demandam processamento intensivo, o uso de *async/await* não traz benefícios significativos, sendo necessário o uso de outras estratégias de paralelismo, como processamento paralelo ou distribuição de carga, dependendo do cenário.

De forma conceitual, o modelo assíncrono pode ser comparado a um atendente que, ao solicitar uma tarefa externa, como a preparação de um pedido, não permanece aguardando sua conclusão de forma ociosa, mas passa a atender outras demandas. Quando o pedido está pronto, ele retorna para concluir a tarefa inicial. Esse comportamento permite melhor aproveitamento do tempo e dos recursos disponíveis ([Microsoft, 2025i](#)).

Dessa forma, o uso de execução assíncrona no ASP.NET Core é um fator fundamental para a escalabilidade da aplicação, permitindo que o servidor atenda um maior número de requisições simultaneamente, otimizando o uso dos recursos disponíveis e reduzindo a necessidade de aumento proporcional de *threads* em execução.

4.6 Finalização da requisição e descarte de recursos

Após a execução do *endpoint* e a conclusão das operações necessárias, a requisição entra em sua fase final, onde a resposta é construída e retornada ao cliente. No ASP.NET Core, esse processo ocorre de forma integrada ao pipeline de *middlewares*, seguindo o fluxo inverso ao percorrido durante o processamento da requisição ([Microsoft, 2025k](#)).

Conforme apresentado anteriormente, cada *middleware* pode executar lógica tanto antes quanto após a chamada do próximo componente da cadeia. Dessa forma, após a execução do *endpoint*, a resposta percorre o pipeline no sentido inverso, permitindo que os *middlewares* realizem operações adicionais, como manipulação da resposta, *logging* ou tratamento de erros finais.

Ao final desse fluxo, a resposta *HTTP* é enviada ao cliente por meio do servidor web, normalmente o *Kestrel*, contendo o resultado da operação solicitada, seja ele um retorno de sucesso, erro ou qualquer outro status definido pela aplicação ([Microsoft, 2025g](#)).

Durante todo o processamento da requisição, conforme apresentado na seção anterior, o ASP.NET Core mantém um escopo de injeção de dependência ativo ([Microsoft, 2025n](#)). Esse escopo é responsável por gerenciar a criação e o ciclo de vida das dependências utilizadas na execução da requisição.

Quando o processamento da requisição é finalizado, o escopo de execução criado pelo ASP.NET Core é automaticamente encerrado pelo *framework*. Esse escopo, que foi responsável por gerenciar todas as dependências utilizadas durante a requisição, deixa de existir, marcando o encerramento do ciclo de vida das instâncias associadas a esse escopo.

Nesse momento, o contêiner de injeção de dependência executa o descarte das instâncias que implementam as interfaces *IDisposable* ou *IAsyncDisposable*, respeitando o tempo de vida configurado para cada serviço ([Microsoft, 2025f](#)). Esse processo ocorre de forma automática e ordenada, garantindo que recursos utilizados pelas dependências sejam liberados corretamente.

Serviços registrados com tempo de vida *Scoped*, por exemplo, têm seu descarte reali-

zado ao final da requisição, uma vez que sua existência está diretamente vinculada ao escopo criado para aquele processamento. Já serviços *Transient* podem ser descartados ao término de sua utilização dentro do escopo, enquanto serviços *Singleton* permanecem ativos durante todo o ciclo de vida da aplicação, sendo descartados apenas no encerramento do *host* ([Microsoft, 2025k](#)).

Esse mecanismo é especialmente relevante em cenários que envolvem o uso de recursos externos, como conexões com banco de dados, arquivos ou chamadas de rede. A liberação adequada desses recursos evita vazamentos de memória, esgotamento de conexões e degradação de desempenho ao longo do tempo.

Dessa forma, o gerenciamento automático do ciclo de vida das dependências pelo contêiner de injeção de dependência é um dos fatores que contribuem para a robustez e estabilidade de aplicações desenvolvidas com ASP.NET Core.

Todo esse controle de descarte é realizado de forma automática pelo framework. Dessa forma, o desenvolvedor não deve invocar manualmente o método *Dispose()* em dependências gerenciadas pelo contêiner, pois esse processo já é controlado internamente.

No contexto da API Ordem, apresentada ao longo deste trabalho, esse processo ocorre automaticamente após a execução do método da *controller*, garantindo que todos os serviços utilizados durante o processamento da requisição sejam devidamente finalizados, sem a necessidade de intervenção manual por parte do desenvolvedor.

Dessa forma, o ciclo de vida de uma requisição em uma API .NET compreende um fluxo completo que se inicia com o recebimento da chamada pelo servidor, passa pelo processamento no pipeline de *middlewares*, pela resolução de dependências e execução do endpoint, e se encerra com o retorno da resposta ao cliente e o descarte dos recursos utilizados.

Esse modelo estruturado garante não apenas a organização do processamento interno da aplicação, mas também a eficiência na utilização de recursos, permitindo que aplicações desenvolvidas com ASP.NET Core sejam altamente escaláveis, resilientes e robustas em cenários de alta concorrência.

4.7 Fluxo completo de uma requisição

Para consolidar os conceitos apresentados ao longo deste capítulo, é possível representar o ciclo de vida de uma requisição em uma API .NET como um fluxo contínuo que envolve diferentes componentes da aplicação, desde o recebimento da chamada até o retorno da resposta ao cliente.

Esse fluxo integra todos os elementos abordados anteriormente, incluindo o servidor web, o pipeline de *middlewares*, a resolução de dependências, a execução do endpoint, o uso de operações assíncronas e o descarte automático de recursos ao final do processamento.

O fluxo completo de uma requisição pode ser descrito pelas seguintes etapas:

1. O cliente envia uma requisição HTTP para a API.
2. O servidor web (*Kestrel*) recebe a requisição.
3. O ASP.NET Core cria o objeto *HttpContext*.
4. A requisição entra no pipeline de *middlewares*.
5. Cada *middleware* processa a requisição e chama o próximo componente da cadeia.
6. O mecanismo de roteamento identifica o *endpoint* correspondente.
7. O contêiner de injeção de dependência resolve as dependências necessárias dentro do escopo da requisição.
8. O *controller* ou *endpoint* é instanciado e executado.
9. Operações assíncronas são realizadas durante a execução (quando aplicável).
10. A resposta é gerada e percorre o *pipeline* de *middlewares* no sentido inverso.
11. O escopo de dependências é encerrado e os recursos são descartados.
12. A resposta *HTTP* é enviada ao cliente.

A figura a seguir ilustra de forma simplificada o fluxo do ciclo de vida de uma requisição no contexto de uma aplicação .NET:

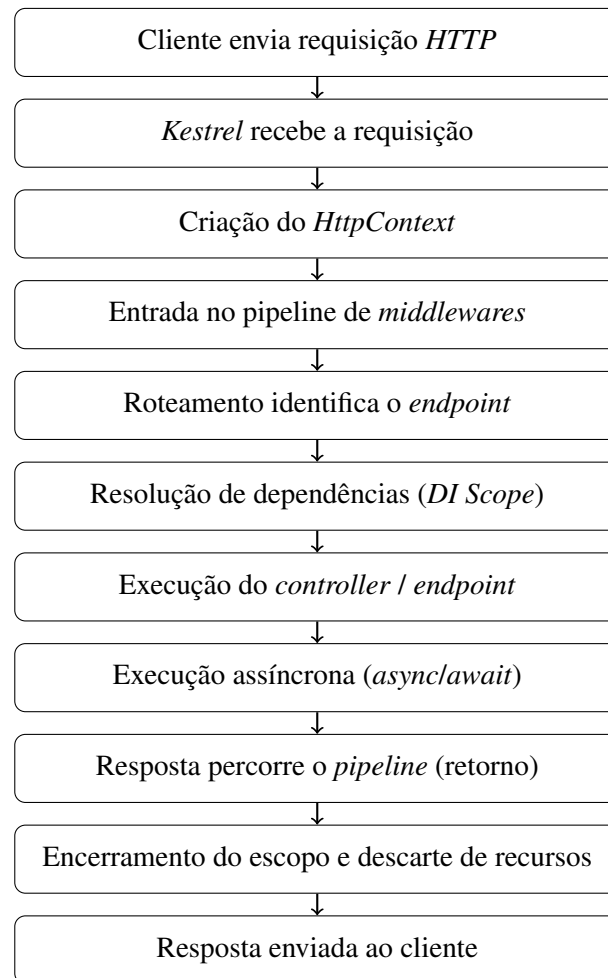


Figura 4 – Fluxo completo do ciclo de vida de uma requisição em uma API .NET.

Dessa forma, é possível visualizar de maneira clara como o processamento da requisição ocorre dentro do contexto da aplicação, evidenciando sua relação com o ciclo de vida da aplicação, no qual está inserido.

Referências

ALMEIDA, R. M. G. d. Boas práticas em desenvolvimento de apis para integração de sistemas. *Lumen et Virtus*, XVI, n. XLV, p. 1636–1655, 2025. Disponível em: <<https://doi.org/10.56238/levv16n45-069>>. Citado na página 5.

Codecademy. *What is HTTP?* 2024. Artigo educacional sobre o protocolo HTTP. Disponível em: <<https://www.codecademy.com/article/what-is-http>>. Citado 2 vezes nas páginas 18 e 19.

MACHADO, R. B. *Estudo sobre os princípios SOLID*. 2025. Documento técnico em formato PDF. Disponível em: <https://www.sunsalesystem.com.br/static/Estudo___SOLID.pdf>. Citado 2 vezes nas páginas 12 e 13.

MACHADO, R. B. *Ordem: Sistema de gestão e controle de processos*. 2026. Aplicação web desenvolvida pelo autor. Disponível em: <<https://ordem.sunsalesystem.com.br/>>. Citado na página 19.

Microsoft. *Hosting model for .NET APIs | Back-end Web Development with .NET for Beginners*. 2023. Vídeo educacional disponível na plataforma Microsoft Learn. Disponível em: <<https://learn.microsoft.com/en-us/shows/back-end-web-development-with-dotnet-for-beginners/hosting-model-for-dotnet-apis-backend-web-development-with-dotnet-for-beginners>>. Citado na página 6.

Microsoft. *AuthorizationMiddleware.Invoke Method*. 2025. Documentação oficial da API do ASP.NET Core. Disponível em: <<https://learn.microsoft.com/en-us/dotnet/api/microsoft.aspnetcore.authorization.authorizationmiddleware.invoke?view=aspnetcore-9.0>>. Citado 2 vezes nas páginas 22 e 24.

Microsoft. *Build web APIs with ASP.NET*. 2025. Disponível em: <<https://dotnet.microsoft.com/en-us/apps/aspnet/apis>>. Citado na página 5.

Microsoft. *Criar APIs Web com ASP.NET*. 2025. Disponível em: <<https://dotnet.microsoft.com/pt-br/apps/aspnet/apis>>. Citado na página 5.

Microsoft. *Dependency injection guidelines in .NET*. 2025. Documentação oficial do .NET. Disponível em: <<https://learn.microsoft.com/en-us/dotnet/core/extensions/dependency-injection/guidelines>>. Citado 3 vezes nas páginas 16, 17 e 27.

Microsoft. *Graceful Shutdown, Linger Options, and Socket Closure*. 2025. Documentação oficial da API Winsock (Windows). Disponível em: <<https://learn.microsoft.com/en-us/windows/win32/winsock/graceful-shutdown-linger-options-and-socket-closure-2>>. Citado na página 10.

Microsoft. *IDisposable.Dispose Method*. 2025. Documentação oficial da API do .NET. Disponível em: <<https://learn.microsoft.com/pt-br/dotnet/api/system.idisposable.dispose?view=netcore-3.1>>. Citado na página 29.

Microsoft. *Kestrel web server in ASP.NET Core*. 2025. Documentação oficial do servidor web Kestrel no ASP.NET Core. Disponível em: <<https://learn.microsoft.com/en-us/aspnet/core/fundamentals/servers/kestrel?view=aspnetcore-10.0>>. Citado 2 vezes nas páginas 20 e 29.

Microsoft. *Middleware no ASP.NET Core*. 2025. Documentação oficial do ASP.NET Core. Disponível em: <<https://learn.microsoft.com/pt-br/aspnet/core/fundamentals/middleware/?view=aspnetcore-10.0>>. Citado na página 22.

Microsoft. *Programação assíncrona com async e await em C#*. 2025. Documentação oficial da linguagem C#. Disponível em: <<https://learn.microsoft.com/pt-br/dotnet/csharp/asynchronous-programming/>>. Citado 3 vezes nas páginas 27, 28 e 29.

Microsoft. *ServiceProvider Class | Microsoft.Extensions.DependencyInjection*. 2025. Documentação oficial da API do .NET. Disponível em: <<https://learn.microsoft.com/en-us/dotnet/api/microsoft.extensions.dependencyinjection.serviceprovider?view=net-10.0-pp>>. Citado na página 6.

Microsoft. *Tempos de vida de serviço na injeção de dependência no .NET*. 2025. Documentação oficial do .NET. Disponível em: <<https://learn.microsoft.com/pt-br/dotnet/core/extensions/dependency-injection/service-lifetimes>>. Citado 5 vezes nas páginas 13, 14, 16, 29 e 30.

Microsoft. *Thread pool gerenciado no .NET*. 2025. Documentação oficial do .NET. Disponível em: <<https://learn.microsoft.com/pt-br/dotnet/standard/threading/the-managed-thread-pool>>. Citado na página 28.

Microsoft. *Use HttpContext in ASP.NET Core*. 2025. Documentação oficial do ASP.NET Core. Disponível em: <<https://learn.microsoft.com/en-us/aspnet/core/fundamentals/use-http-context?view=aspnetcore-10.0>>. Citado na página 20.

Microsoft. *Visão geral da injeção de dependência no .NET*. 2025. Documentação oficial do .NET. Disponível em: <<https://learn.microsoft.com/pt-br/dotnet/core/extensions/dependency-injection/overview>>. Citado 3 vezes nas páginas 12, 24 e 29.

Microsoft Tech Community. *Anatomia de um serviço ASP.NET Core*. 2024. Artigo publicado na Microsoft Tech Community. Disponível em: <<https://techcommunity.microsoft.com/blog/desenvolvedoresbr/anatomia-de-um-servi%C3%A7o-asp-net-core/4423980>>. Citado 2 vezes nas páginas 18 e 19.

NIZZOLA, M. *Injeção de dependência em C#: entendendo Singleton, Scoped e Transient*. 2023. Artigo técnico publicado no Medium. Disponível em: <<https://marcionizzola.medium.com/inje%C3%A7%C3%A3o-de-depend%C3%A4ncia-em-c-entendendo-singleton-scoped-e-transient-80f79b0b6d68>>. Citado 2 vezes nas páginas 15 e 16.

Postman, Inc. *Postman: API Platform for Building and Using APIs*. 2025. Plataforma para desenvolvimento, teste e documentação de APIs. Disponível em: <<https://www.postman.com/>>. Citado na página 19.

Refactoring.Guru. *Singleton*. 2025. Material educacional sobre padrões de projeto. Disponível em: <<https://refactoring.guru/pt-br/design-patterns/singleton>>. Citado 2 vezes nas páginas 3 e 14.